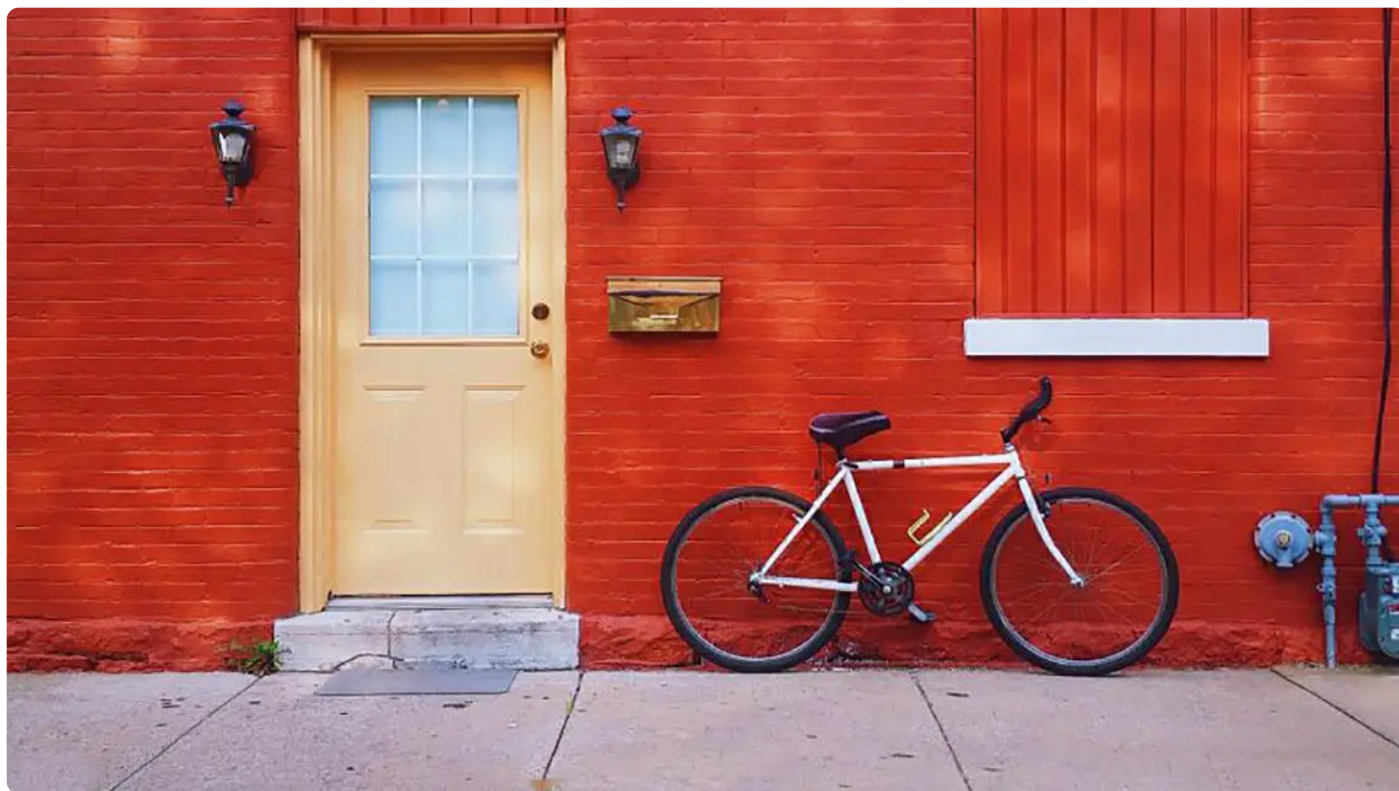


## 02 | 所有权（上）：Rust是如何管理程序中的资源的？

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们来讲讲 Rust 语言设计的出发点——所有权，它也是 Rust 的精髓所在。

在第一节课中，我们了解了 Rust 语言里的值有两大类：一类是固定内存长度（简称固定尺寸）的值，比如 `i32`、`u32`、由固定尺寸的类型组成的结构体等；另一类是不固定内存长度（简称非固定尺寸）的值，比如字符串 `String`。这两种值的本质特征完全不一样。而**怎么处理这两种值的差异，往往是语言设计的差异性所在。**

就拿数字类型来说，C、C++、Java 这些语言就明确定义了数字类型会占用内存中的几个字节，比如 8 位，也就是一个字节，16 位，也就是两个字节。而 JavaScript 这种语言，就完全屏蔽了底层的细节，统一用一个 `Number` 表示数字。Python 则给出了 `int` 整数、`float` 浮点、`complex` 复数三种数字类型。

Rust 语言因为在设计时就定位为一门通用的编程语言（对标 C++），它的应用范围很广，从最底层的嵌入式开发、OS 开发，到最上层的 Web 应用开发，它都要兼顾。所以它的数字类

型不可避免地就得暴露出具体的字节数，于是就有了 i8、i16、i32、i64 等类型。

前面我们说到，一种类型如果具有固定尺寸，那么它就能够在编译期做更多的分析。实际上固定尺寸类型也可以用来管理非固定尺寸类型。具体来说，Rust 中的非固定尺寸类型就是靠指针或引用来指向，而指针或引用本身就是一种固定尺寸的类型。

## 栈与堆

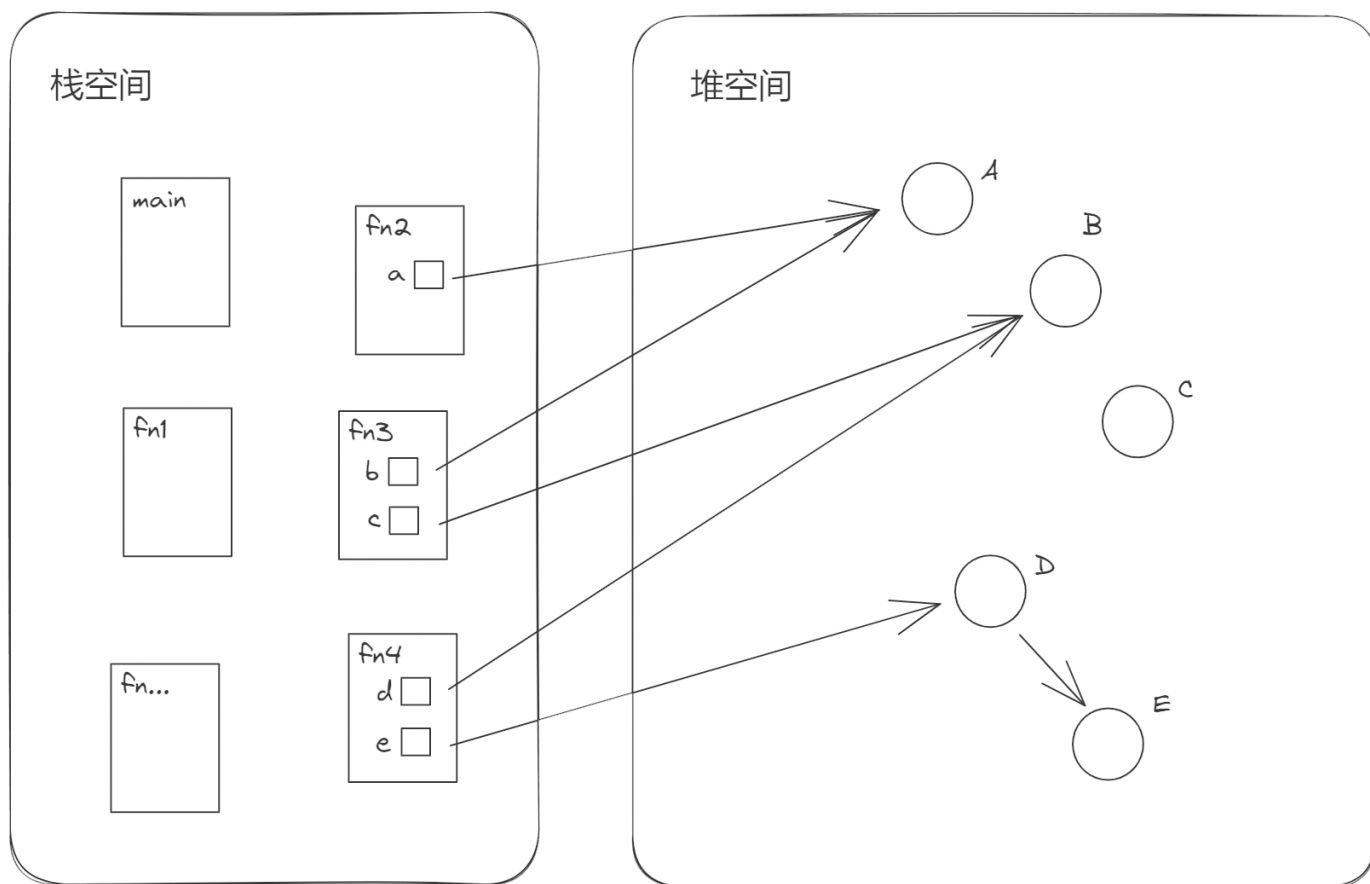
现代计算机会把内存划分为很多个区。比如，二进制代码的存放区、静态数据的存放区、栈、堆等。

栈上的操作比堆高效，因为栈上内存的分配和回收只需移动栈顶指针就行了。这就决定了分配和回收时都必须精确计算这个指针的增减量，因此**栈上一般放固定尺寸的值**。另一方面，栈的容量也是非常有限的，因此也不适合放尺寸太大的值，比如一个有 1000 万个元素的数组。

那么非固定尺寸的值怎么处理呢？在计算机体系架构里面，专门在内存中拿出一大块区域来存放这类值，这个区域就叫“堆”。

## 栈空间与堆空间

在一般的程序语言设计中，栈空间都会与函数关联起来。每一个函数的调用，都会对应一个帧，也叫做 frame 栈帧，就像图片栈空间里的方块 main、fn1、fn2 等。一个函数被调用，就会分配一个新的帧，函数调用结束后，这个帧就会被自动释放掉。因此**栈帧是一个运行时的**事物。函数中的参数、局部变量之类的资源，都会放在这个帧里面，比如图里 fn2 中的局部变量 a，这个帧释放时，这些局部变量就会被一起回收掉。



栈空间与堆空间

函数的调用会形成层级关系，因此栈空间中的帧可能会同时存在很多个，并且在它们之间也对应地形成层级关系。如上图所示，可能的函数调用关系为，main 函数中调用了函数 fn1，fn1 中调用了函数 fn2，fn2 中调用了函数 fn3，fn3 中调用了函数 fn4，fn4 调用了更深层次的其他函数。这样的话，在程序执行的某个时刻，main 函数、fn1、fn2、fn3、fn4 等对应的帧副本就同时存在于栈中了。

图中右边堆空间里面的一些小圈表示堆空间中资源，也就是被分配的内存。从图中可以看到，栈空间中函数帧的局部变量是可以引用这些堆上资源的。一个栈帧中的多个局部变量可以指向堆中的多个资源，如 fn3 中的 b 指向资源 A，c 指向资源 B；同时存在的多个栈帧中的局部变量还可以指向堆上的同一个资源，如图中的 a 和 b，c 和 d；堆上的资源也可以存在引用关系，如图中的 D 和 E。

如果一个资源没有被任何一个栈帧中的变量引用或间接引用，如图中的 C，那么它实际是一个被泄漏的资源，也就是发生了内存泄漏。被泄漏的资源会一直伴随程序的运行，直到程序自身

的进程被停止时，才会一起被 OS 回收掉。

而计算机程序内存管理的复杂性，主要就在于**堆内存的管理比较复杂——既要高效，又要安全**。

这里我们稍微提及了一点计算机的结构知识，你可以停下来仔细理解这张图示表达的意思，在后面我们还会经常回顾这张图。有了栈和堆的知识作为铺垫，你会更容易理解 Rust 中的一些特性为什么要那样设计。

下面我们回到 Rust 语言，继续讲 Rust 中另一个重要概念——可变性。

## 变量与可变性

回顾第一讲的知识，在 Rust 中定义一个变量，使用 `let variable = value;` 这种语法。比如 `let x = 10u32;`，就定义了变量 `x`。然后，`10u32` 是一个值，它被绑定到这个变量上。

默认变量是不可变的，我们来做个实验。

[复制代码](#)

```
1 fn main() {
2     let x = 5;
3     println!("The value of x is: {x}");
4     x = 6;
5     println!("The value of x is: {x}");
6 }
```

输出：

[复制代码](#)

```
1 $ cargo run
2   Compiling variables v0.1.0 (file:///projects/variables)
3 error[E0384]: cannot assign twice to immutable variable `x`
4   --> src/main.rs:4:5
5   |
6 2 |         let x = 5;
```

```
7 | -
8 | |
9 | first assignment to `x`
10 | help: consider making this binding mutable: `mut x`
11 3 | println!("The value of x is: {x}");
12 4 | x = 6;
13 | ^^^^^ cannot assign twice to immutable variable
```

Rust 默认这样做是为了减少一些很低级的 Bug。假如默认可以改的话，如果你在一个代码量很大而且离定义变量很远的某个分支语句里面修改了这个变量的值，然后在后面某个函数调用里面又用到了它，结果导致程序行为与期望不符，这时你很难看出来问题出在哪儿。这种低级错误能不犯就不犯，Rust 干脆帮你禁用了这种方式。

但是下面这样做是可以的。

[复制代码](#)

```
1 fn main() {
2     let x = 5;
3     println!("The value of x is: {x}");
4     let x = 6;    // 注意这里，重新使用了 let 来定义新变量
5     println!("The value of x is: {x}");
6 }
```

这种方式在 Rust 中叫做变量的 Shadowing。意思很好理解，就是定义了一个新的变量名，只不过这个变量名和老的相同。原来那个变量就被遮盖起来了，访问不到了。这种方式最大的用处是程序员不用再去费力地想另一个名字了！变量的 Shadow 甚至支持新的变量的类型和原来的不一样。

比如：

 复制代码

```
1 fn main() {
2     let a = 10u32;
3     let a = 'a';
4     println!("{}", a);
5 }
```

那如果我们要修改变量的值应该怎么做呢？只需要在变量名前面加一个 `mut` 就可以声明一个变量为可以修改内容的。

 复制代码

```
1 let mut x = 10u32;
```

例子：

 复制代码

```
1 fn main() {
2     let mut x = 5;
3     println!("The value of x is: {x}");
4     x = 6;
5     println!("The value of x is: {x}");
6 }
7 // 输出
8 The value of x is: 5
9 The value of x is: 6
```

注意，值的改变只能在同一种类型中变化，在变量 `x` 定义的时候，就已经确定了变量 `x` 的类型为数字了，你可以试试将其改成字符串，看会报什么错误。

这里你可以回过头去对比一下，可修改变量和变量的 `Shadow` 的不同之处。

一个变量，其内容是否可变，被称作这个变量的**可变性**（mutability）。`mut` 叫作可变性修饰符（modifier）。

可能你会非常疑惑，变量不就应该会变化的吗？既然默认不可变，为什么要称其为变量呢？其实上面一段我已经回答了这个问题，Rust 中变量的可变性是一种潜力，只要它有可能会变化，那么就可以称之为变量。而 Rust 给这种潜力加了一道开关，**当你想让这个变量的可变性暴露出来的时候，就在变量名前面明确地加个 mut 修饰符。**

可以看到，变量名加了 mut，多打了 4 个字符，这实际是在代码中留下了一种足迹。也就是说给了程序员一个信息，当你自己或别的程序员在读到这个变量的定义时，他会知道，后面一定会修改这个变量，因为如果你后面没修改它，Rust 编译器会提示你把这个 mut 去掉。

这种设计还有一个好处，那就是减少滥用概率。我们在这里构造一个编程语言界的墨菲定律，**如果一个特性不太利于程序的健壮性，但是很好用，滥用的成本非常低，那么它一定会被滥用。**

比如 TypeScript 中的 any 类型，有时写 TS 代码懒得去设计类型，直接就用 any 类型了，反正“先跑通了再说”。结果就是最后项目完成了，代码里面 any 满天飞，TS 的设计初衷被抛至脑后。偷懒是人的天性，Rust 接受了这种天性，让你想要修改一个变量的时候，需要多付出点成本，也就是多打 4 个字符。

另一个例子是 JS 中的 var 和 let，都是三个字符，敲的字符数一样，成本一样，结果就是在语言层面并不能驱动程序员往好的实践方面靠。有人会辩称，在这些语言中会有推荐规范或强制要求，要求你按好的实践方式写。不过在实际项目中，由于进度等问题，这些规范总是很难完全贯彻下去，即使贯彻下去也很难达到预期效果，这方面已有太多案例了。因为那些都是补救措施，哪有从语言层面强制约束你做来得统一。

## 变量的类型

值是有类型的，比如 10u32，它就是一个 u32 类型的数字。一旦一个变量绑定了一个值，或者说一个值被绑定到了一个变量上，那么这个变量就被指定为这种值的类型。比如 `let x = 10u32;` 编译器会自动推导出变量 x 的类型为 u32。完整的写法就是 `let x: u32 = 10u32;`。

此外还有一种方式，就是直接先指定变量的类型，然后把一个值绑定上去，比如 `let x: u32 = 10;`。这种方式更好，**它能说明你在写这句代码的时候就已经对它做了一个架构上的**

**规划和设计**，这种形式能帮助我们在编译阶段阻止一些错误。

比如输入下面这段代码：

```
1 fn main() {  
2     let a: u8 = 323232;  
3     println!("{a}");  
4 }
```

[复制代码](#)

编译器就会报错，指出 u8 类型装不下这么大的一个数字。

```
1 error: literal out of range for `u8`  
2 --> src/main.rs:5:17  
3 |  
4 5 |     let a: u8 = 323232;  
5   |                   ^^^^^^^  
6 |  
7   = note: the literal `323232` does not fit into the type `u8` whose range is `0..
```

[复制代码](#)

看到这个错误，你是不是感觉 Rust 特别贴心。同样的代码，你可以放在其他语言中实现，做一下对比。

**所有的变量都应该具有明确的类型是 Rust 程序的基本设计。**当然其他语言中也有类型，不同语言对类型重视的程度不一样，这取决于语言自身的设计定位。

好了，变量的概念我们先解析到这里，下面我们来看一个 Rust 中的“奇怪”行为。

## Rust 中“奇怪”的行为

我们先来看一个例子。

```
1 fn main() {
```

[复制代码](#)

```
2     let a = 10u32;
3     let b = a;
4     println!("{a}");
5     println!("{b}");
6 }
```

很简单，它打印出：

```
1 10
2 10
```

[复制代码](#)

然后我们再来看字符串的行为，你猜一下程序会输出什么。

```
1 fn main() {
2     let s1 = String::from("I am a superman.");
3     let s2 = s1;
4     println!("{s1}");
5     println!("{s2}");
6 }
```

[复制代码](#)

是两行 “I am a superman” 吗？反正在其他语言中是这样的。

结果在 Rust 中不是，编译器给出了出错信息，我们来看看。

```
1     Compiling playground v0.0.1 (/playground)
2 error[E0382]: borrow of moved value: `s1`
3 // 借用了移动后的值 `s1`
4 --> src/main.rs:4:15
5 |
6 2 |     let s1 = String::from("I am a superman.");
7   |           -- move occurs because `s1` has type `String`, which does not impleme
8 // 移动发生了，因为 `s1` 的类型是 `String`，而这种类型并没有实现 `Copy` trait。
9 3 |     let s2 = s1;
10  |           -- value moved here
11 // 在这里值移动了。
```

[复制代码](#)

```

12 4 |     println!("{s1}");
13   |           ^^^^^ value borrowed here after move
14 // 值在被移动后在这里被借用
15   |
16   = note: this error originates in the macro `crate::format_args_nl` which comes
17 help: consider cloning the value if the performance cost is acceptable
18 // 如果性能成本可以接受的话，考虑克隆这个值
19   |
20 3 |     let s2 = s1.clone();
21   |           ++++++++

```

既然给出了修改建议，那我们直接照着代码建议改一下试试。

[复制代码](#)

```

1 fn main() {
2     let s1 = String::from("I am a superman.");
3     let s2 = s1.clone();
4     println!("{s1}");
5     println!("{s2}");
6 }

```

好了，这下输出我们预期的结果了。

[复制代码](#)

```

1 I am a superman.
2 I am a superman.

```

Rust 中的字符串为何有如此奇怪的行为呢？

## 所有权

首先，我们看到在 Rust 中，字符串的行为好像与 u32 这种数字类型不一样。前面我们说过，u32 这种类型是固定尺寸类型，而 String 是非固定尺寸类型。一般来说，对于固定尺寸类型，会默认放在栈上；而非固定尺寸类型，会默认创建在堆上，成为堆上的一个资源，然后在栈上用一個局部变量来指向它，如代码中的 s1。

在将一个变量赋值给另一个变量的时候，不同语言对底层细节的处理不一样。这里我们拿 Java 举例。前面我们说过，局部变量都是定义在栈帧中的，Java 也是一样。Java 语言对于 int 这类固定尺寸类型，在复制给另一个变量的时候，会直接复制它的值。在面对 Object 这种复杂对象的时候，默认只会复制这个 Object 的引用给另一个变量。这个引用的值（内存地址）就存在栈上的局部变量里面。

为什么会这样设计呢？因为如果那个 Object 占用的内存很大，每一次重新赋值，就把那个对象重新拷贝一次，也就是完全克隆，是非常低效的，凡是脑筋正常的语言都不会那样干。所以在用 Java 编程时，它实际上是隐藏了对象**引用的复制**这个细节。

回到 Rust，我们看到对于 u32 这种固定尺寸类型来说，Rust 与 Java 也是同样的处理，直接在栈上进行内容的拷贝。而对于字符串这种动态长度的类型来说，在变量的再赋值上，Rust 除了拷贝字符串的引用外，实际还做了更多事情。具体是什么事情呢？我们先来看一下修改后的例子。

[复制代码](#)

```
1 fn main() {  
2     let s1 = String::from("I am a superman.");  
3     let s2 = s1;  
4     //println!("{s1}");  
5     println!("{s2}");  
6 }
```

这个例子，就能正常打印。

[复制代码](#)

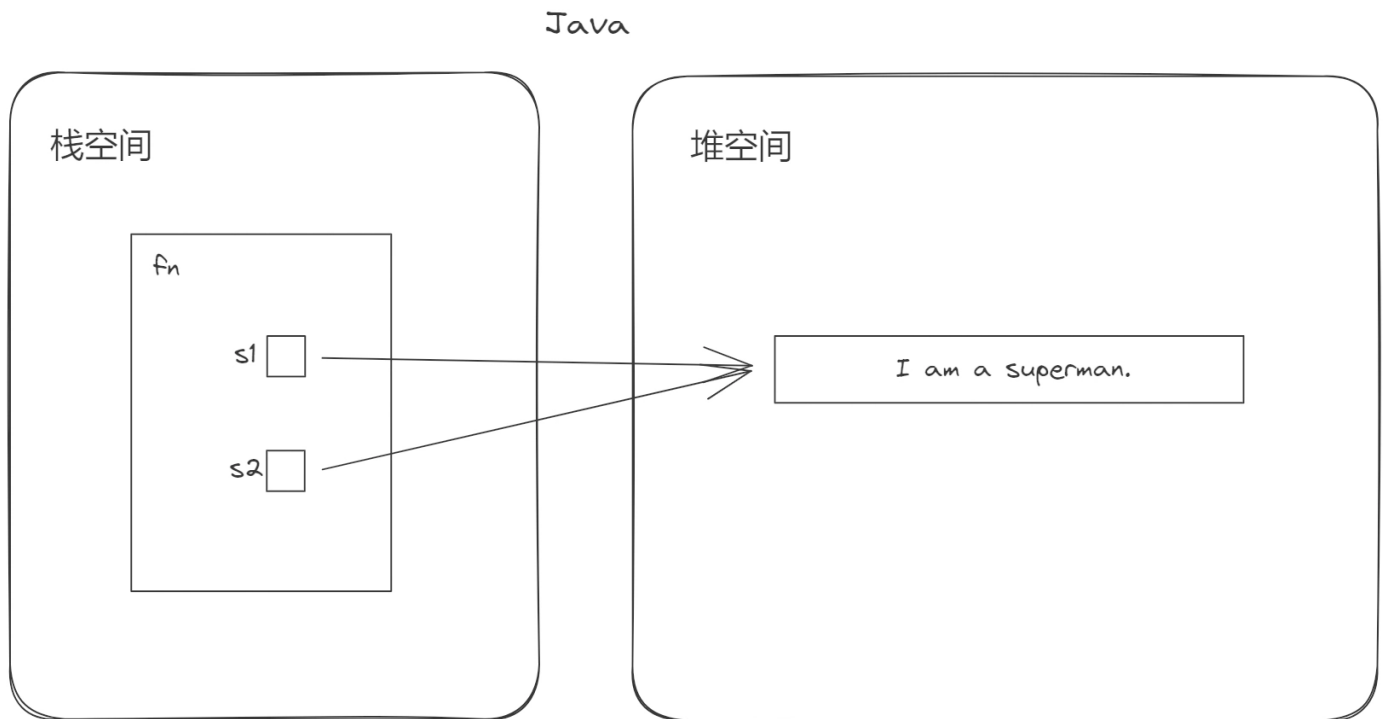
```
1 I am a superman.
```

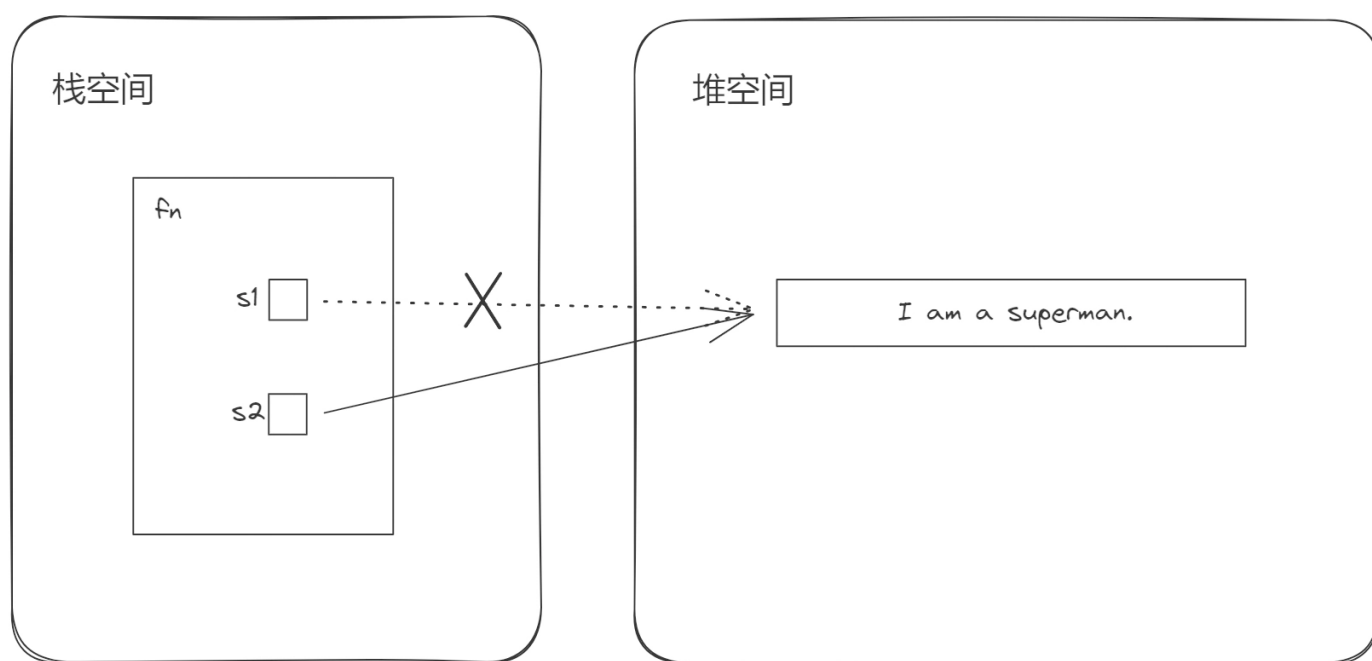
对比之后，我们发现 `let s2 = s1;` 语句执行后，s2 可以使用，而 s1 不能再使用了。也就是说，在 Rust 里面，s1 把内容“复制”给 s2 后，s2 可用，s1 不能用了！

从代码层面我们也可以说，s1 把值（资源）“移动”给了 s2。既然是移动了，那原来的变量就没有那个值了。请仔细体会这里与 Java 的不同之处。Java 默认做了引用的拷贝，并且新旧

两个变量同时指向原来那个对象。而 Rust 不一样，**Rust 虽然也是把字符串的引用由 s1 拷贝到了 s2，但是只保留了最新的 s2 到字符串的指向，同时却把 s1 到字符串的指向给“抹去”了。**s1 之后都处于一种“不可用”的状态，直到函数结束。这就是 Rust 编译器做的那个“更多”的部分。

下面的图示展示了这两种行为上的差异。





好奇怪呀！Rust 怎么会这样设计呢？

其实这正是 Rust 从头开始梳理整个软件体系的地方，剑指一个目标：**内存安全**。

## 所有权

长久以来，计算机领域最聪明的大脑都在探索如何写出更安全的程序，为此建立了各种理论、模式、模型。而 Rust 不走寻常路，它采用了一种全新的思路，利用所有权来管理内存资源，保证内存安全。接下来我们就一起来好好品鉴一下这个独特的思路。此刻，**请你先卸下之前固有的思维，将脑袋放空一下。**

Rust 明确了所有权的概念，值也可以叫资源，所有权就是拥有资源的权利。一个变量拥有一个资源的所有权，那它就要负责那个资源的回收、释放。**Rust 基于所有权定义出发，推导出了整个世界。**

所有权的基础是三条定义。

1. Rust 中，每一个值都有一个所有者。

2. 任何一个时刻，一个值只有一个所有者。
3. 当所有者所在作用域（scope）结束的时候，其管理的值会被一起释放掉。

这三条规则涉及两个概念：**所有者和作用域**。

所谓所有者，在代码里就用变量表示。而变量的作用域，就是变量有效（valid）的那个代码区间。在 Rust 中，一个所有权型变量的作用域，简单来说就是它定义时所在的那个最里层的花括号括起的部分，从变量创建时开始，到花括号结束的地方。

比如：

 复制代码

```
1 fn main() {
2     let s = String::from("hello");
3     // do stuff with s
4 } // 变量s的作用域到这里结束
5
6 fn main() {
7     let a = 1u32;
8     {
9         let s = String::from("hello");
10    } // 变量s的作用域到这里结束
11    // xxxx
12
13 } // 变量a的作用域到这里结束
```

变量在其作用域内是有效的，离开作用域就无效了。

好，理解了这一点，我们现在尝试用所有权规则去翻新一下对前面例子的理解。

```
1 fn main() {  
2     let a = 10u32;  
3     let b = a;  
4     println!("{a}");  
5     println!("{b}");  
6 }
```

在这个例子中，a 具有对值 10u32 的所有权。执行 `let b = a` 的时候，把值 10u32 复制了一份，b 具有对这个新的 10u32 值的所有权。当 main 函数结束的时候，a、b 两个变量就离开了作用域，其对应的两个 10u32，就都被回收了。这里是栈帧结束，栈帧内存被回收，局部变量位于栈帧中，所以它们所占用的内存就被回收了。

再来看一个字符串的例子。

```
1 fn main() {  
2     let s1 = String::from("I am a superman.");  
3     println!("{s1}");  
4 }
```

局部变量 s1 拥有这个字符串的所有权。s1 的作用域从定义到开始，直到花括号结束。s1（栈帧上的局部变量）离开作用域时，变量 s1 上绑定的内存资源（字符串）就被回收掉了。注意，这里发生的事情是，栈帧中的局部变量离开作用域了，顺带要求堆内存中的字符串资源被回收。之所以能够做到这一点，是因为这个堆中的字符串资源被栈帧中的局部变量所指向了的。

而从 Rust 的语法层面看起来，就是变量 s1 对那个字符串拥有所有权。所以 s1 离开作用域的时候，那个资源就一起被回收了。这看起来好像是一个自动的过程，我们并没有像 C 语言中那样，需要手动调用 `free()` 函数去释放堆中的字符串资源。

这种**堆内存资源随着关联的栈上局部变量一起被回收**的内存管理特性，叫作 RAII（Resource Acquisition Is Initialization）。它实际不是 Rust 的原创，而是 C++ 创造的。如果你学过 C 的话，可以对比一下 C 中的 `malloc()` 分配堆内存的方式，在分配堆内存后，C 语言里面必须

由程序员手动在后续的代码中使用 `free()` 来释放堆内存中的资源。而有了 RAII 特性后，我们不需要手动写 `free()`，因此可以认为 RAII 内存管理方式是一个相当大的进步。

有了所有权的知识后，我们再回过头来分析上面那个例子。

 复制代码

```
1 fn main() {  
2     let s1 = String::from("I am a superman.");  
3     let s2 = s1;  
4     //println!("{s1}");  
5     println!("{s2}");  
6 }
```

变量 `s1` 持有这个字符串的所有权。`s1` 对字符串的所有权从第 2 行定义时开始，到 `let s2 = s1` 执行后结束。这一行执行后，`s2` 持有那个字符串的所有权。而此时 `s1` 处于什么状态呢？处于一种不可用的状态，或者叫无效状态（invalid），这个状态是由 Rust 编译器在编译阶段帮我们管理的，我们只需要从所有权模型去理解它，而不需要操心细节。Rustc 小助手把这些事情给我们打理得明明白白的。

然后直到花括号结束，`s2` 及 `s2` 所拥有的字符串内存，就被回收掉了，`s1` 所对应的那个局部变量的内存空间也一并被回收了。

**所有权是 Rust 语言的出发点，我们写的任何 Rust 程序，都必须遵循这套规则。**

需要注意的一点是，所有权其实是内存结构之上的更上层概念，并不是说只有在堆中分配的资源才有所有权。实际上，栈上的资源也是有所有权的。所有权这个概念实际上屏蔽了底层内存结构的细节，让我们可以站在一个新的层次上更有效地对问题进行建模。

这个思维一定要注意，Rust 语言中并不是所有的分析都需要归结到内存结构上去才能搞清楚，思维一直停留在内存结构上，有时会妨碍你的抽象建模能力，就像你精通量子力学不一定能当一个好的建筑师，所以这一点尤其要注意。

## 使用所有权书写函数

下面我们来看一下，基于所有权规则，函数的写法会变成什么样。

 复制代码

```
1 fn foo(s: String) {  
2     println!("{s}");  
3 }  
4  
5 fn main() {  
6     let s1 = String::from("I am a superman.");  
7     foo(s1);  
8 }
```

输出：

 复制代码

```
1 I am a superman.
```

没问题。

稍微改动一下例子，我们想在函数调用结束后，在外面再打印一下 s1 的值。

 复制代码

```
1 fn foo(s: String) {  
2     println!("{s}");  
3 }  
4  
5 fn main() {  
6     let s1 = String::from("I am a superman.");  
7     foo(s1);  
8     println!("{s1}");    // 这里加了一行  
9 }
```

咦，编译出错了。提示：

 复制代码

```
1 error[E0382]: borrow of moved value: `s1`
```

```

2  --> src/main.rs:8:16
3  |
4  6 |      let s1 = String::from("I am a superman.");
5  |          -- move occurs because `s1` has type `String`, which does not impleme
6  7 |      foo(s1);
7  |          -- value moved here
8  8 |      println!("{s1}");
9  |          ^^ value borrowed here after move
10 |
11 note: consider changing this parameter type in function `foo` to borrow instead i
12 --> src/main.rs:1:11
13 |
14 1 | fn foo(s: String) {
15 |     ---      ^^^^^^^ this parameter takes ownership of the value
16 |     |
17 |     in this function
18 = note: this error originates in the macro `$crate::format_args_nl` which comes
19 help: consider cloning the value if the performance cost is acceptable
20 |
21 7 |      foo(s1.clone());
22 |          ++++++++

```

这个例子在其他语言中，一般是不会有问题的。foo 函数也许会修改字符串的值，在外面重新打印的时候，会打印出新的值。但是，这其实是一种相当隐晦的设计模式，可能会造成一些错误（在下一讲我们会讲到），而 Rust 阻止了这种模式。

这个例子代码的提示与前面差不多，就是说 s1 所有权已经被移动进函数里面了，不能在移动后再使用了。

注意提示中的这一行：

 复制代码

```

1 1 | fn foo(s: String) {
2   |     ---      ^^^^^^^ this parameter takes ownership of the value

```

函数的参数 s 获取了这个值的所有权。函数参数是这个函数的一个局部变量，它在这个函数栈帧结束的时候会被回收，因此这个字符串在这个函数调用结束后，就已经被回收了，这就是我们无法再打印这个字符串的原因。

同样我们再看一个上面例子的变形。

[复制代码](#)

```
1 fn foo(s: String) {  
2     println!("{s}");  
3 }  
4  
5 fn main() {  
6     let s1 = String::from("I am a superman.");  
7     foo(s1);  
8     foo(s1);  
9 }
```

我们简单地想调用两次 `foo()` 函数都做不到，原因跟前面是一样的。这就是 Rust 有点反直觉的地方，也是令很多初学者崩溃的地方。原因我们再重复一下，一个苹果，你给了别人，那你就没有了。一个知识，我教给了你，我们都会得到。Rust 的编程模型默认选择了前者，而以往的主流编程语言默认选择了后者。

回到前面例子，那我们后面的代码还想用 `s1`，该怎么办？

可以这样，既然能把所有权移动到函数里面，也当然能把所有权转移出来。

[复制代码](#)

```
1 fn foo(s: String) -> String {  
2     println!("{s}");  
3     s  
4 }  
5  
6 fn main() {  
7     let s1 = String::from("I am a superman.");  
8     let s1 = foo(s1);  
9     println!("{s1}");  
10 }
```

这样就输出了结果：

```
1 I am a superman.  
2 I am a superman.
```

[📄 复制代码](#)

我们适配了 Rust 的所有权规则，实现了我们期望的函数调用效果。

## 移动还是复制

前面讲到，u32 这种类型在做变量的再赋值的时候，是做了复制所有权的操作。而 String 这种类型在做变量再赋值的时候，是做了移动所有权的操作。那么，在 Rust 中哪些类型默认是做移动所有权操作，哪些类型默认是做复制所有权操作呢？

默认做复制所有权的操作的有 7 种。

所有的整数类型，比如 u32；

布尔类型 bool；

浮点数类型，比如 f32、f64；

字符类型 char；

由以上类型组成的元组类型 tuple，如 (i32, i32, char)；

由以上类型组成的数组类型 array，如 [9; 100]；

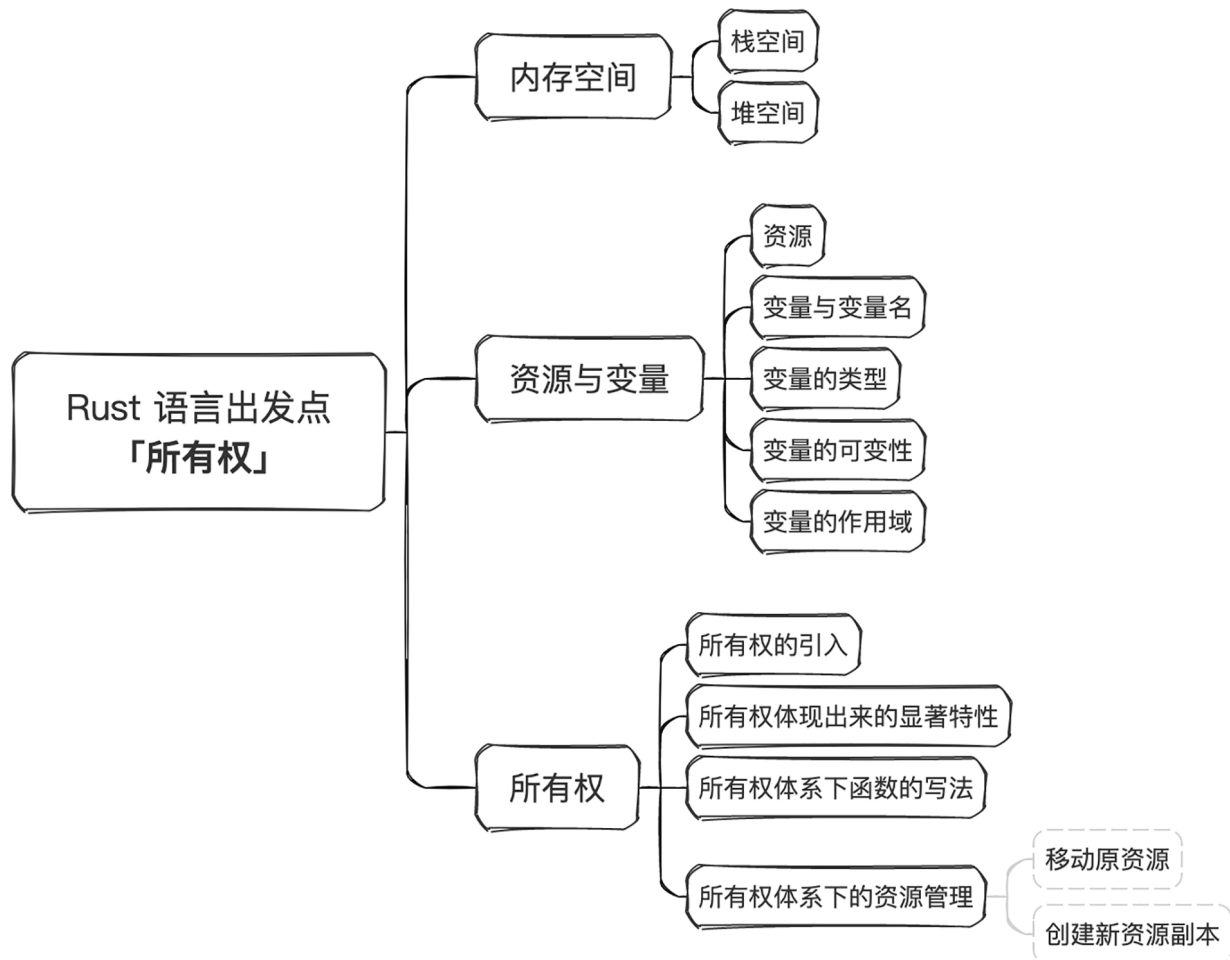
不可变引用类型 &。

其他类型默认都是做移动所有权的操作。

## 小结

所有权是 Rust 语言中非常重要的一个概念，用于**管理程序中使用的资源**。这些资源可以是堆上的动态分配的内存资源，也可以是栈上的内存资源，或者是其他的系统资源，比如 IO 资源。所有权通过把语句绑定在变量上，封装了栈和堆的实现细节。对于固定尺寸基础类型（小尺寸类型），它们的值默认是可复制的，这主要是为了编程方便。对于非固定尺寸类型或大尺寸类型的变量再赋值时，默认使用移动操作。除非显式地 clone，否则它只保持一份所有权。

所有权可以被转移，一旦所有权被转移，原来持有该资源的变量就失效了。变量的作用域是在最近的花括号位置内。



## 思考题

最后我来考一考你。

1. 下面的示例将输出什么？

```
1 fn main() {
2     let s = "I am a superman.".to_string();
3
4     for i in 1..10 {
5         let tmp_s = s;
```

```
6         println!("{}", tmp_s);  
7     }  
8 }
```

2. 一个由固定尺寸类型组成的结构体变量，如下面示例中的 Point 类型，在赋值给另一个变量时，采用的是移动方式还是复制方式？

[复制代码](#)

```
1 struct Point {  
2     x: i64,  
3     y: i64,  
4     z: i64  
5 }
```

欢迎你把你思考后的答案分享到评论区，和我一起讨论，也欢迎你把这节课分享给需要的朋友，邀他一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 全部留言 (37)

最新 精选



**二夕Thrower**

2023-10-23 来自浙江

思考题 1: 无法通过编译，可以将第 5 行代码修改为：let tmp\_s = s.clone();

思考题 2: 由于 Point 没有实现 Copy trait，所以在赋值过程中会产生 Move。

作者回复: 真棒！

共 2 条评论 >

16



**Geek\_147053**

2023-10-24 来自北京

看完了刚更新的前2篇，感觉挺适合新手的，虽然长，但看下来一点儿也不枯燥，讲的挺有意思

编辑回复：加油！一直跟下去，不会让你失望的



👍 5



**Forest**

2023-10-25 来自四川

思考题 1：编译报错；两处错误：变量 i 没有使用和 s 的所有权；

修改后如下：

```
fn main() {  
    let s = "I am a superman.".to_string();  
  
    for _ in 1..10 {  
        let tmp_s = s.clone();  
        println!("s is {}", tmp_s);  
    }  
}
```

思考题 2：移动；如果结构体中包含实现了 Copy trait，则会进行复制而不是移动

作者回复：对的。

共 2 条评论 >

👍 3



**Geek\_582a5d**

2023-10-24 来自上海

目前看来感觉看着最舒服的rust相关系列文章了，催催更新。这个系列会持续跟进，感谢作者。

编辑回复：如果觉得有帮助的话，欢迎转发给身边的朋友哦，你一票我一票 Mike 老师要出道！



👍 3



**古明地觉**

2023-11-01 来自北京

好奇一个问题：

```
fn main() {  
    let s1 = "hello world".to_string();  
    let s2 = s1; // s1 不再有效，它的值发生了移动  
}
```

显然上面代码很好理解，如果换一种方式。

```
fn main() {  
    let arr = ["hello world".to_string()];  
    let s = arr[0]; // 报错：String 没有实现 Copy trait  
}
```

这里报错了，按理说不是应该会将所有权从 arr[0] 转移到 s 上面吗？但是 Rust 却提示 cannot move。我的理解是这样的，因为像数组这样的结构如果有效，那么它内部的每一个成员必须都要有效。如果数组中的某个元素发生了移动，那么会导致整个数组不可用，于是为了避免这种情况，Rust 要求数组里面的元素必须是可 Copy 的。如果需要转移所有权，那么 Rust 编译器就报错。否则会给用户造成一个错觉，好端端的数组为啥就不能用了。

如果用这个理论来解释的话，那么就又产生了一个问题。

```
fn main() {  
    // 元素  
    let arr = ("hello world".to_string(), "hello world".to_string());  
    let s = arr.0; // 此处不报错  
    // arr.1 可以正常打印  
    println!("{}", arr.1); // hello world  
    // 但打印 arr.0 和打印 arr 则报错，提示发生了移动  
}
```

所以我很好奇，为啥会产生这种情况。为什么元组（还有结构体）允许局部的元素发生移动，但数组却不可以呢？还请老师帮忙解答一下

作者回复：思考得非常棒。我的理解是结构体，元组这种属于异质复合结构，每个元素都有单独的类型指定。array，Vec属于同质连续结构。可能语言设计者认为后者比前者更紧致，随意置某个位置无效的话会引起更多麻烦，比如sort操作咋办？迭代操作咋办？估计是有这方面的考虑从而禁止把所有权从数组和动态数组中移动出来。

共 2 条评论 >

👍 2



Onion

2023-10-25 来自北京

默认做复制所有权的操作：

裸指针类型

裸指针类型也是copy语义吧

作者回复：默认做复制所有权的操作：

裸指针类型。

这句话从哪里来的？文章里面没有。

共 2 条评论 >

👍 2



咖啡 ☕

2023-10-24 来自上海

问题 1

改成

```

let tmp\_s = &s;

```

即可

作者回复：👍



👍 2



duwoodly

2023-10-24 来自重庆

1. String类型，实际数据在堆上存储。let tmp\_s = s 循环第一次的时候，会移动所有权，s在栈上的内存虽然还在，但是被编译器视为无效变量或无效状态，所以第二次及以后的循环就不能实验变量s了，编译器会报错。

2. 结构体类型默认没有实现Copy trait, 赋值过程也会移动所有权。

当然从底层看，这个Point结构体的成员都是基本类型（基本类型实现了Copy），所以这个结构体的值是保存在栈上的，所以赋值操作，实际上底层是在栈上完整拷贝了一次Point结构体，但是编译器依然会把原来的Point结构体变量视为无效状态或无效变量。

作者回复：👍 很棒



👍 2



刘永臣

2023-10-24 来自北京

问题1：本来以为坑在s 因为s在第一次遍历时已经转移了，所以第二次遍历肯定会出错，所以编译器会报错，没有问题；但是实际上循环条件中i 会被多次修改，所以声明i时也应该增加mut；

问题2：结构体数据复合结构，所以对其进行赋值是实际上进行了所有权的转移。

作者回复：👍



👍 2



^ ^

2023-11-23 来自中国香港

所以说上面那些默认使用copy复制所有权的变量类型，其实都是rust为他们实现了copy trait的是吗

作者回复：对的，正解。因为这些太基础了太常用了，而且是固定尺寸的。



👍 1